

FORMULARIOS WEB

Los formularios web facilitan la comunicación entre un usuario y un sitio web o aplicación.

Los formularios permiten a los usuarios la introducción de datos, que generalmente se envían a un servidor web para su procesamiento y almacenamiento.

Los controles pueden ser *campos de texto* de una o varias líneas, *cajas desplegables*, *botones*, *casillas de verificación* o *botones de opción*, y se crean principalmente con el elemento `<input>`

o Diseñar el formulario

Antes de comenzar a escribir el código, hay que tomarse el tiempo necesario para pensar en el formulario. Diseñar una maqueta rápida te ayudará a definir el conjunto de datos que deseas que el usuario introduzca. Desde el punto de vista de la experiencia del usuario (UX), es importante saber que si pides muchos datos te arriesgas a frustrar a las personas y perder usuarios. Tiene que ser simple y conciso: solicita sólo los datos que realmente necesitas.

Vamos a crear un formulario de contacto sencillo:

Una maqueta de un formulario de contacto web. El formulario está encerrado en un recuadro con una sombra y esquinas redondeadas. Contiene cuatro elementos: un campo de texto para el nombre, un campo de texto para el correo electrónico, un área de texto para el mensaje y un botón que dice 'Enviar mensaje'.

Este formulario va a tener tres campos de texto y un botón. Le pediremos al usuario su nombre, su correo electrónico y el mensaje que desea enviar. Al pulsar el botón sus datos se enviarán a un servidor web.

Creemos ya el HTML para nuestro formulario. Vamos a utilizar los siguientes elementos HTML: `<form>`, `<label>`, `<input>`, `<textarea>` y `<button>`.

Dentro del body de nuestro documento HTML crearemos el elemento `<form>` que es un contenedor específico para contener formularios:

```
<form action="/pagDestinoServidor" method="post">
```

```
</form>
```

Todos sus atributos son opcionales, pero normalmente se establece siempre al menos los atributos **action** y **method**

- **action**: contiene la ubicación de la página (URL) a la que se enviarán los datos que se han recopilado en el formulario.
- **method**: define la manera de enviar los datos al servidor. Los valores posibles son:
 - **get**: los valores enviados se añaden a la dirección indicada en el atributo **action**
 - **post**: los valores se envían de forma separada, y no se ven en la url

Los elementos <label> <input> <textarea>

Nuestro formulario de contacto es sencillo: la parte para la entrada de datos contiene tres campos de texto, cada uno con su elemento **<label>** correspondiente:

- El campo de entrada para el *nombre* es un campo de texto con una sola línea
- El campo de entrada para el *email* es un campo de texto de tipo email (acepta solo direcciones de correo electrónico) y de una sola línea.
- El campo de entrada para el *mensaje* es **<textarea>**: un campo de texto multilínea.

En términos de HTML, para implementar estos controles de formulario necesitamos algo como esto:

```
<form method="get" action="/pagServidor">
  <div>
    <label for="nom">Nombre:</label>
    <input type="text" id="nom" name="nombre"/>
  </div>
  <div>
    <label for="mail">Correo electrónico:</label>
    <input type="email" id="mail" name="email"/>
  </div>
  <div>
    <label for="mens">Mensaje:</label>
    <textarea id="mens" name="mensaje"></textarea>
  </div>
  <div>
    <button type="submit">Enviar mensaje</button>
  </div>
</form>
```

Por motivos de usabilidad y accesibilidad incluimos una etiqueta (*label*) para cada control de formulario. Fíjate en el atributo **for** de todos los elementos **<label>**, verás que toma como valor el **id** del control de formulario con el que está asociado, así es como asociamos un control con su etiqueta.

Hacer esto tiene muchas ventajas porque la etiqueta está asociada al control del formulario y permite que los usuarios con ratón, panel táctil y dispositivos táctiles hagan

clic en la etiqueta para activar el control correspondiente, y también proporciona accesibilidad con un nombre que los lectores de pantalla leen a sus usuarios.

En el elemento `<input>`, el atributo más importante es **type**. Este atributo es muy importante porque define la forma en que el elemento `<input>` aparece y se comporta (luego veremos más información sobre esto)

- En nuestro ejemplo usamos `input con type=text` para el nombre. Representa un campo de texto básico de una sola línea que acepta cualquier tipo de entrada de texto.
- Para la segunda entrada, usamos `input con type=email`, que define un campo de texto de una sola línea que solo acepta una dirección de correo electrónico. Esto convierte un campo de texto básico en una especie de campo “inteligente” que realiza algunas comprobaciones de validación de los datos que el usuario escribe. También hace que aparezca un diseño de teclado más apropiado para introducir direcciones de correo electrónico (por ejemplo, con un símbolo @ por defecto) en dispositivos con teclados dinámicos, como teléfonos inteligentes.

Para el campo de mensaje usamos el elemento `<textarea>`, en él también se introduce texto, pero como ves su sintaxis es distinta a la de `<input>` porque éste al ser un elemento vacío, no necesita etiqueta de cierre. Esto implica que Para definir el valor predeterminado de un elemento `<input>`, debemos usar el atributo **value** de la siguiente manera:

```
<input type="text" value="Este campo se llena por defecto con este texto">
```

En cambio, si queremos definir un valor predeterminado para un elemento `<textarea>`, lo debemos colocar entre las etiquetas de apertura y cierre del elemento `<textarea>`, así:

```
<textarea>
  Por defecto, este elemento contiene este texto
</textarea>
```

El elemento `<button>`

Ya tenemos casi completo el formulario, sólo falta añadir un botón que se encargue de mandar los datos una vez se haya rellenado el formulario.

Esto se hace con el elemento `<button>`, así que añade lo siguiente justo encima de la etiqueta de cierre `</form>`

```

<div>
  <button type="submit">Enviar mensaje</button>
</div>
```

El elemento `<button>` también acepta un atributo **type**, que a su vez acepta uno de estos tres valores: `submit`, `reset` o `button`.

- Un clic en un botón `submit` (el valor predeterminado) envía los datos del formulario a la página web definida por el atributo `action` del elemento `<form>`.
- Un clic en un botón `reset` restablece todos los controles de formulario a su valor por defecto. Desde el punto de vista de UX, esto se considera una mala práctica, así que hay que evitar este tipo de botones a menos que haya una buena razón para incluirlos.
- Un clic en un botón `<button>` no hace nada, pero es muy útil para crear botones personalizados y se le puede dar funcionalidad con JavaScript.

También se puede usar el elemento `<input>` con el atributo *type* correspondiente para generar un botón, por ejemplo `<input type="submit">`. La ventaja principal del elemento `<button>` es que el elemento `<input>` solo permite texto sin formato en su etiqueta, mientras que el elemento `<button>` permite contenido HTML completo, lo que permite generar botones creativos más complejos.

Botón de imagen

El control *botón de imagen* se muestra exactamente como un elemento `<img>`, excepto que cuando el usuario hace clic en él, se comporta como un botón de envío.

Se crea un botón de imagen usando un elemento `<input>` con su atributo *type* establecido en el valor `image`. Este elemento admite los mismos atributos que el elemento `<img>` y todos los atributos que admiten el resto de botones de formulario.

```
<input type="image" alt="Descarga"
src="boton.jpg" width="140px" height="50px" >
```



Si queremos que el formulario tenga mejor aspecto, podemos crear una hoja de estilos con estas clases:

```
<style>
  form {
    margin: 0 auto;
    width: 400px;
    padding: 1em;
    border: 1px solid #7d91db;
    border-radius: 1em;
  }
  form div {
    margin: 5px;
  }
  label {
    display: inline-block;
    width: 90px;
    text-align: left;
  }
  input,
```

```

textarea {
  font-family: 'Courier New', Courier, monospace;
  font-size: 1em;
  width: 300px;
  border: 2px solid #c82306;
  border-radius: 7px;
}
input:focus,
textarea:focus {
  border-color: black;
}
button {
  border: #c82306 solid 2px;
  border-radius: 5px;
  background-color: #c82306;
  color: white;
  padding: 5px;
  width: 300px;
}
button:hover {
  background-color: #FFFFFF;
  color: #c82306;
}
</style>

```

Enviar los datos del formulario a un servidor web

La última parte y seguramente la más complicada es manejar los datos del formulario en el lado del servidor. El elemento `<form>` define dónde y cómo enviar los datos gracias a los atributos `action` y `method`.

A cada control de formulario le damos un nombre (`name`). Los nombres son importantes para que tanto el navegador como el servidor puedan identificar los datos que se envían. Los datos del formulario se envían al servidor como pares de nombre/valor.

Para poner nombre a los diversos datos que se introducen en un formulario, se usa el atributo `name` en cada control de formulario que recopila un dato específico.

Si miramos el formulario, vemos que el formulario manda 3 datos denominados *nom*, *mail* y *mens* a la url `/pagDestinoServidor`, mediante el método *POST*.

En el lado del servidor, se reciben los datos contenidos en la solicitud HTTP. La forma en que se manejan esos datos depende del lenguaje de servidor elegido para manipular estos datos (PHP, Python, Ruby, Java, C#, etc.)

Vamos a ver ahora en detalle algunos de los controles que se suelen usar en los formularios

Campos de entrada de texto

Los campos de texto `<input>` son los controles de formulario más básicos. Son un modo sencillo de permitir al usuario introducir cualquier tipo de datos.

Todos los controles de texto básicos tienen algunas características comunes:

- Se pueden marcar como **readonly** (el usuario no puede modificar el valor de entrada, pero este se envía con el resto de los datos del formulario)
- **disabled** (el valor de entrada no se puede modificar y nunca se envía con el resto de los datos del formulario).
- Pueden tener un **placeholder** se trata de un texto que aparece dentro de la caja de entrada de texto y que se usa para describir brevemente el propósito de la caja de texto.
- Se les puede poner una limitación de tamaño (el tamaño físico de la caja de texto) con **size** y de la extensión máxima **maxlength**(el número máximo de caracteres que se pueden poner en la caja de texto)

Campos de texto de una sola línea

Un campo de texto de una sola línea se crea utilizando un elemento `<input>` cuyo valor de atributo **type** se establece en `text`, u omitiendo por completo el atributo **type** (text es el valor predeterminado).

Campos de contraseña

El valor de la contraseña no añade restricciones especiales al texto, pero oculta el valor que se introduce en el campo (por ejemplo, con puntos o asteriscos) para impedir que otros puedan leerlo.

```
<input type="password" name="pass" >
```

Campos número de teléfono

```
<input type="tel" id="telefono" name="telefono">
```

Cuando se accede desde un dispositivo táctil con teclados dinámicos, muchos de ellos mostrarán un teclado numérico cuando se encuentren con `type="tel"`

Campo URL

```
<input type="url" id="url" name="url">
```

Este tipo añade restricciones de validación en el campo. El navegador informará de un error si no se introdujo el protocolo (como `http:`), o si de algún modo el URL está mal formado. En dispositivos con teclados dinámicos a menudo mostrará por defecto algunas o todas las teclas como los dos puntos, el punto, y la barra inclinada.

Campo numérico

Este control se parece a un campo de texto pero solo permite números de punto flotante, y normalmente proporciona botones deslizadores para incrementar o reducir el valor del control. En dispositivos con teclados dinámicos generalmente se muestra el teclado numérico.

Con el tipo de `input number` puedes limitar los valores mínimo y máximo permitidos definiendo los atributos **min** y **max**.

También puedes utilizar el atributo `step` para cambiar el incremento y decremento causado por los botones deslizadores. Por defecto, el tipo de input number sólo valida si el número es un entero. Para permitir números decimales, especifica `step="any"`. Si se omite, su valor por defecto es 1, lo que significa que solo son válidos números enteros.

```
<input type="number" name="age" id="age" min="1" max="10" step="2">
```

Campos fecha

```
<input type="datetime-local" name="datetime" id="datetime">
```

```
<input type="month" name="month" id="month">
```

```
<input type="time" name="time" id="time">
```

```
<input type="week" name="week" id="week">
```

```
<label for="myDate">¿Cómo caen las vacaciones de Navidad?</label>
```

```
<input type="date" name="myDate" min="2020-12-22" max="2021-01-08" step="7" id="fecha">
```

Elementos de selección: casillas de verificación y botones de opción

Los elementos de selección son controles cuyo estado puede cambiar cuando se hace clic en ellos o en sus etiquetas asociadas. Hay dos tipos de elementos de selección:

- las casillas de verificación (o checkbox) y
- los botones de opción (o radiobutton).

Ambos usan el atributo `checked` para indicar si el control de formulario está seleccionado por defecto o no.

Estos controles no se comportan de la misma manera que otros controles de formulario. Para la mayoría de los controles de formulario, cuando se envía el formulario, se envían todos los controles que tienen un atributo `name`, incluso si en ellos no se ha introducido ningún valor. En cambio, con elementos de selección, sus valores se envían sólo si están seleccionados. Si no están seleccionados, no se envía nada.

Las *casillas de verificación* se crean con un elemento `<input>` estableciendo su atributo `type = checkbox`.

```
<input type="checkbox" id="acepto" name="acepto" value="Sí" checked>
```

Al incluir el atributo `checked`, la casilla de verificación se marca automáticamente cuando se carga la página. Al hacer clic en la casilla de verificación o en su etiqueta asociada, la casilla de verificación se activa o desactiva.

El botón de opción se crea con un elemento `<input>` estableciendo su atributo `type = radio`.

```
<input type="radio" id="edad" name="edad" value="18" checked>
```

Es posible asociar diversos botones de opción, si comparten el mismo valor de atributo `name`, se considera que están en el mismo grupo de botones. Solo puede estar activado en cada momento un botón dentro de un grupo. Esto significa que cuando uno de ellos se selecciona, todos los demás se deseleccionan automáticamente:

```
<fieldset>
  <legend>¿Cuál es tu comida favorita?</legend>
  <ul>
    <li>
      <label for="sopa">Sopa</label>
      <input type="radio" id="sopa" name="comida" value="sopa"
        checked>
    </li>
    <li>
      <label for="curry">Curry</label>
      <input type="radio" id="curry" name="comida" value="curry">
    </li>
    <li>
      <label for="pizza">Pizza</label>
      <input type="radio" id="pizza" name="comida" value="pizza">
    </li>
  </ul>
</fieldset>
```

Cajas de selección (combobox y listbox)

La etiqueta HTML `select` se utiliza para crear menús desplegables de los que los usuarios pueden seleccionar la opción que deseen. Las opciones a elegir se codifican dentro de la etiqueta `<option>`.

Para crear un **combobox** escribiríamos dentro de un formulario:

```
<select name="Cine">
  <option>Ábaco</option>
  <option>Conde Duque</option>
  <option selected="selected">Kinépolis</option>
  <option>Palacio de hielo</option>
  <option>Yelmo</option>
</select>
```


Con esto veremos al entrar en la página:



Una de las opciones aparece seleccionada por defecto con el atributo: `selected="selected"`



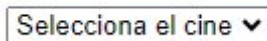
y si abrimos el combo tendremos:

Pero ésta no es una buena experiencia de usuario ya que éste podría enviar una opción no deseada con solo pulsar el botón de envío del formulario.

Podemos mejorarla añadiendo en la primera posición la opción "Selecciona el cine":

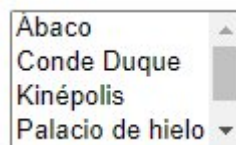
```
<select name="Cine">
  <option selected="selected">Selecciona el cine</option>
  <option>Ábaco</option>
  <option>Conde Duque</option>
  <option>Kinépolis</option>
  <option>Palacio de hielo</option>
  <option>Yelmo</option>
</select>
```

Y ahora veremos:



Si ahora queremos crear un **listbox**, sólo tenemos que añadir el atributo `size` que nos indica cuántos elementos de la lista se verán simultáneamente (en los combobox, por defecto `size=1`)

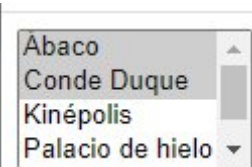
```
<select name="Cine" size="4">
  <option>Ábaco</option>
  <option>Conde Duque</option>
  <option>Kinépolis</option>
  <option>Palacio de hielo</option>
  <option>Yelmo</option>
</select>
```



De esta manera veremos en el navegador:

También podemos permitir seleccionar más de una opción usando el atributo `multiple`:

```
<select name="Cine" size="4" multiple="multiple">
  <option>Ábaco</option>
  <option>Conde Duque</option>
  <option>Kinépolis</option>
  <option>Palacio de hielo</option>
  <option>Yelmo</option>
</select>
```



Tanto en uno como en otro podemos usar el elemento **optgroup** que nos permite agrupar las distintas opciones en categorías:

```
<select name="Cine">
  <option selected="selected">Selecciona el cine</option>
  <optgroup label="En Madrid">
    <option>Ábaco</option>
    <option>Conde Duque</option>
    <option>Kinépolis</option>
  </optgroup>
  <optgroup label="En Sevilla">
    <option>CineZona</option>
    <option>Los Arcos</option>
  </optgroup>
</select>
```

Y así al desplegar el combo, veremos:



Atributos comunes

Muchos controles de formulario tienen sus atributos específicos propios. Sin embargo, hay un conjunto de atributos que son comunes para todos los elementos de formulario:

Nombre del atributo	Valor por defecto	Descripción
<u>autofocus</u>	false	Este atributo booleano permite especificar que el elemento ha de tener el foco de entrada automáticamente cuando se carga la página. En un documento, solo un elemento asociado a un formulario puede tener este atributo especificado.
<u>disabled</u>	false	Este atributo booleano indica que está deshabilitado, es decir, el usuario no puede interactuar con el elemento. Si no se especifica este atributo, el elemento hereda su configuración del elemento que lo contiene, por ejemplo, <code><fieldset></code>. Si el elemento que lo contiene no tiene el atributo establecido en <i>disabled</i>, el elemento está habilitado.
<u>name</u>		El nombre del elemento; se envía con los datos del formulario.
<u>value</u>		El valor inicial del elemento.